Reinforcement Learning for Bio-Inspired Torque-Based Quadruped Locomotion: A Cross-Simulator Reproduction and Analysis

Li Chuan-Han

Department of Biomechatronics Engineering, National Taiwan University

June 2026

Contents

Abstract

This report is a **deep reinforcement learning** project that trains and analyzes a torque-based quadruped locomotion controller using neural-network policies, Proximal Policy Optimization (PPO), and ablation studies. Most RL controllers use **position control** (outputting target joint angles that a low-level PD controller converts to torque): easy to train, but stiff, insufficiently compliant, and prone to overreacting to disturbances. **Torque control** commands joint torques directly and is more compliant, but its highly nonlinear action space and inefficient early exploration have long made it hard to train. We fully reproduce **Safe and Adaptive Torque-Based Locomotion Policies Inspired by Animal Learning (SATA)** (arXiv:2502.12674), which "shapes" the torque action space with three bio-inspired mechanisms (activation low-pass, Hill force-velocity model, motor fatigue) plus a developmental growth curriculum, making it both easier to learn and more physically feasible.

The implementation has two stages. On Isaac Gym we reproduce SATA (eight seeds, reward 103.9 ± 15.9) and analyze it via per-mechanism ablation, a hardware-feasibility evaluation of all forty-eight policies under payload and push disturbances, and a classical residual term. The core finding is counter-intuitive: ablations that *raise* training reward in a clean simulator do so by *leaving the hardware-feasible*

range—disabling activation drives peak torque to 42.5 N·m (near the real Go2's 45 N·m limit), and disabling fatigue uses 2.5× energy and 35× action jerk. The bio mechanisms thus act as sim-to-real feasibility constraints, not reward devices. The second stage migrates the pipeline to NVIDIA Isaac Lab, holding the robot fixed as the control variable: a faithful port reproduces SATA's per-step task performance (positive task terms within about 0.002 per step). All results were trained, evaluated, and reproduced by us; both projects are open-sourced. We maintain statistical rigor throughout (at least five seeds per condition, mean ± sample std, Welch's t-test) and leave unconfirmed phenomena (e.g., trainability) as open questions rather than overclaiming.

Keywords: Deep Reinforcement Learning, Neural Network Policy, PPO, Torque Control, Bio-Inspired Locomotion, Quadruped, Cross-Engine Reproduction

---

Chapter 1. Introduction

1.1 Motivation

Enabling quadruped robots to walk robustly over unknown, unstructured terrain is one of the central problems in embodied AI. Most current reinforcement-learning locomotion controllers output **target joint angles**, which must then pass through a low-level controller to be converted into torques. The advantage of this position control scheme is stable training; its drawback is stiff behavior: it only ever tries to "track the commanded angle," even when the environment indicates it should yield. For example, a leg that has become stuck is still forced toward its target angle, which limits compliance and dynamic behavior and tends to overreact to sudden disturbances.

Torque control offers the opposite characteristics: by commanding joint torques directly, the robot can "absorb" external forces and recover, much like real muscle. The cost, however, is that it is hard to train. The SATA paper states plainly that the action space of torque control is "highly nonlinear" and that "exploration is inefficient during training," which has made prior torque-based reinforcement learning results fragile and sensitive to hyperparameters.

This difficulty is not a vague claim; it can be understood mechanistically. Position control implicitly carries a **stability safety net**: once the policy outputs a target angle, the PD controller drives the joint toward that angle, and the PD restoring force itself provides basic postural stability, so that even a random early policy does not immediately diverge. Torque control has no such protection – the policy commands torque directly, and random early exploration easily produces violent, divergent actions (the robot instantly topples or twitches), from which little useful learning signal can accumulate. Moreover, the mapping from "torque" to "the robot's next state" is far more indirect and nonlinear than from "target angle": the same torque has wildly different consequences at different joint velocities and contact states, making the reward landscape rougher and riddled with local optima. This is precisely where SATA intervenes.

The core claim of SATA is this: rather than brute-forcing the training, one should **"shape" the torque action space using biomechanics**, making it simultaneously easier to learn and more physically feasible.

1.2 Objectives

This report does not propose a new method; it is an implementation-focused reproduction and analysis study with three objectives:

1. **Fully reproduce** SATA's torque control pipeline in simulation (Isaac Gym + Unitree Go2).
2. **Analyze how the bio-inspired design gives rise to adaptive behavior** by ablating the biomechanical model, the fatigue feedback, and the growth mechanism one at a time, and quantifying the role of each.
3. **Migrate across engines** by porting the entire pipeline to Isaac Lab and testing whether "a faithful port still holds after the engine is swapped."

1.3 Contribution and Positioning

The value of this study lies in **hands-on implementation and honest analysis**, not in any claim to overturn or surpass the original paper. Specifically: we connect the ablation results through questions posed by control theory, quantifying the role of the bio-inspired mechanisms along the axis of **hardware feasibility**; we maintain statistical rigor throughout; and where evidence is insufficient, we honestly leave a phenomenon as an open question. Both projects are open-sourced as evidence of inspectable, repeatable research capability.

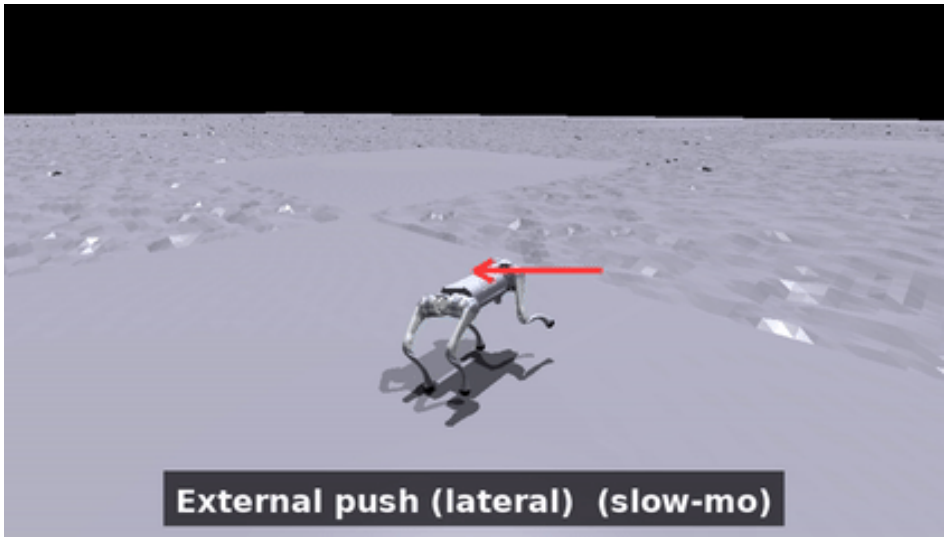

Figure 1. The instant the SATA reference policy absorbs a lateral external force (from `ev_push.gif`) – the robot staggers, regains its footing, and recovers its gait, intuitively demonstrating the compliance of torque control. The arrows are annotations overlaid after the fact; the external force is genuinely applied in simulation.

Chapter 2. Method: Reinforcement Learning and PPO

The core method of this project is **reinforcement learning (RL)** – more precisely, training a neural-network policy with **Proximal Policy Optimization (PPO)**. This chapter explains the RL problem formulation, how PPO works, and the network, observation, action, reward, and hyperparameter settings actually used in this study.

2.1 The Reinforcement Learning Problem

Quadruped locomotion control can be cast as a **Markov Decision Process (MDP)**: at each time step the agent (the robot policy) observes the current **state**, outputs an **action**, the environment transitions to the next state and returns a **reward**. The learning objective is to find a **policy** π – a function mapping states to actions – that maximizes the expected long-term discounted reward.

We use RL rather than a hand-designed control law because the dynamics of a quadruped on rough terrain are highly nonlinear and hard to model analytically; RL lets the policy **learn directly from large amounts of interaction with the simulated environment**, without writing down control equations in advance. In this project the state is the robot's proprioception (velocities, orientation, joint states, etc.), the action is the torque command for the twelve joints, and the reward encourages tracking velocity commands while maintaining body height and posture (see 2.4).

2.2 The PPO Algorithm

PPO is one of the most widely used **policy-gradient** algorithms for continuous control and adopts an **actor-critic** architecture:

- The **actor (policy network)** outputs a probability distribution over actions (here a diagonal Gaussian whose mean is produced by the network and whose standard deviation is a learnable parameter);
- The **critic (value network)** estimates the value of a state, which is used to compute the **advantage** – how much better a given action is than average. We compute advantages with **Generalized Advantage Estimation (GAE, $\lambda = 0.95$, $\gamma = 0.99$)**.

PPO's key innovation is the **clipped surrogate objective**: it limits how far the new-to-old policy probability ratio may deviate from 1 (clip range $\varepsilon = 0.2$) on each update, preventing a single update from being so large that it destroys the learned policy. The loss is

```
L = clipped_surrogate(ε=0.2) + value_loss_coef × value_loss − entropy_coef × entropy
```

where the entropy term (coefficient 0.01) encourages exploration and avoids premature convergence. We also use an **adaptive learning rate**: based on the measured KL divergence each iteration compared with a target (desired_kl = 0.01), the learning rate is doubled or halved to keep updates within a stable range.

> Why PPO: it strikes a good balance between sample efficiency and stability under massively parallel simulation, and it is the algorithm used by the original SATA work – reproducing with the same algorithm lets us attribute any differences to the environment or design rather than to the algorithm itself.

2.3 Network Architecture and Training Scale

- **Network**: both actor and critic are three-layer multilayer perceptrons (MLPs) with hidden layers `[512, 256, 128]` and ELU activations; both share the same 60-dimensional observation (no privileged information). The actor outputs a 12-dimensional Gaussian action (initial standard deviation 1.0). About 400k parameters in total.
- **Training scale**: 4096 parallel environments; each iteration collects 24 steps × 4096 environments $\approx$ 98,000 transitions; each iteration performs 5 epochs × 4 mini-batches = 20 gradient updates. Trained for 3000 iterations, roughly 295 million environment steps. Other hyperparameters: value-loss coefficient 1.0, gradient-norm clip 1.0.

2.4 Observation, Action, and Reward Design

Observation (60-dim) – the robot's proprioception, concatenated as follows:

Index	Content
0:3	Base linear velocity (body frame)
3:6	Base angular velocity
6:9	Projected gravity vector (tilt sensing)
9:21	Joint angle − default angle (12 DOF)
21:33	Joint angular velocity (12 DOF)
33:36	Velocity command (v_x, v_y, ω_yaw)
36:48	Applied joint torques (12 DOF)
48:60	Per-DOF fatigue state (SATA-specific)

Action (12-dim): torque commands for the twelve joints (applied after the bio-inspired layer of Chapter 3).

Reward function – nine terms (positive terms reward task achievement, negative terms penalize undesirable behavior):

Term	Weight	Role
forward	+10	Track forward velocity v_x (exponential of error)
head_height	+5	Maintain body height and stay upright
moving_y	+5	Track lateral velocity v_y
moving_yaw	+5	Track yaw rate ω_yaw
soft_dof_pos_limits	−5	Avoid sitting near mechanical joint limits
motor_fatigue	−0.05	Small penalty on accumulated fatigue

Term	Weight	Role
dof_acc	-1×10^{-6}	Action smoothness (squared joint acceleration)
roll	−5	Avoid tipping sideways
lin_vel_z	−5	Avoid vertical bouncing

This reward design embodies a core trade-off in RL control: the positive terms define "the desired behavior" (walk correctly), while the negative terms constrain "the cost paid" (do not thrash, do not exceed limits). The ablation analysis in later chapters examines precisely how these terms interact with the bio-inspired mechanisms.

Chapter 3. Core Technology: Torque Control and the Bio-Inspired Locomotion Layer

The twelve-dimensional action output by the policy network is not directly equal to the joint torques. At each physics sub-step, the action passes through the following processing (the code resides in `_compute_torques` within `go2_torque.py` of the SATA source). The torque-limit baseline is a uniform 23.5 N·m per degree of freedom (the simulation clipping value).

3.1 The Four Stages of the Torque Processing Pipeline

Stage 1: Action scaling. The dimensionless action output by the policy is multiplied by a scaling factor (action_scale = 5), amplifying it to the torque scale.

Stage 2: Muscle activation low-pass filter (Activation). A first-order exponential moving average (EMA) is applied to the torque signal, updated as "60% from the current command, 40% from the previous step," after first squashing it into the range [-1, 1] with a tanh. The tanh serves two purposes at once: smoothing (modeling the delay of muscle recruitment and avoiding instantaneous jumps of torque between the positive and negative extremes), and a torque ceiling (locking the torque within physical limits). In addition, the system applies a domain randomization called "action dropout" with probability 0.1, which at each step holds the activation at its previous value with that probability.

Stage 3: Hill force-velocity model (Hill model). The formula is

```
torque = activation × torque_limit × (1 − sign(activation) × joint_angular_velocity / angular_velo
```

Its mechanism is an inverse force-velocity relationship: when the joint moves at high speed in the same direction, the available torque drops; in the decelerating direction it is unconstrained. This design derives from Hill's (1938) muscle force-velocity law – the faster a muscle contracts, the less force it can produce. SATA borrows this relationship so the policy cannot forcibly command maximum torque at high speed; it is therefore a "physically grounded saturation limit" rather than an arbitrary clip.

Stage 4: Fatigue update (Motor fatigue). Each degree of freedom maintains a leaky accumulator:

```
fatigue = (fatigue + |torque| × dt) × 0.9
```

Its effective time constant is roughly ten physics steps (about 50 ms at 200 Hz). The fatigue state is fed back into the observation vector, so the policy "perceives its own level of exertion" and learns to distribute load. The reward penalty on fatigue is deliberately set very small (−0.05); the goal is not to forbid exertion, but to gently nudge the policy toward low-fatigue solutions when there is no tracking cost.

3.2 Growth Curriculum

SATA drives a "development level" scalar G(t) between zero and one along a Gompertz curve:

```
G(t) = exp(−exp(−k × (steps − x0)))      # k = 3×10⁻⁵, x0 = 24000
```

Here "steps" refers to cumulative environment steps (not PPO iterations). This single scalar schedules five things at once: control frequency 100 → 200 Hz, front-leg torque limit 0.3 → 1.0×, rear-leg torque held at 1.0× (always at full strength), the command range growing from small to large, and the strength of domain randomization (the magnitude of the extrapolated push force).

The concept of this mechanism is "**embodiment growth**," not a curriculum of task difficulty: it progressively unlocks the capabilities of the robot's **body itself**, so that early training takes place in an "easy-to-control juvenile," thereby avoiding the local optima that torque-based reinforcement learning tends to fall into during a cold start. One notable asymmetry is that in the juvenile stage the front legs are weaker (torque limit only 0.3×), while the rear legs remain at full strength.

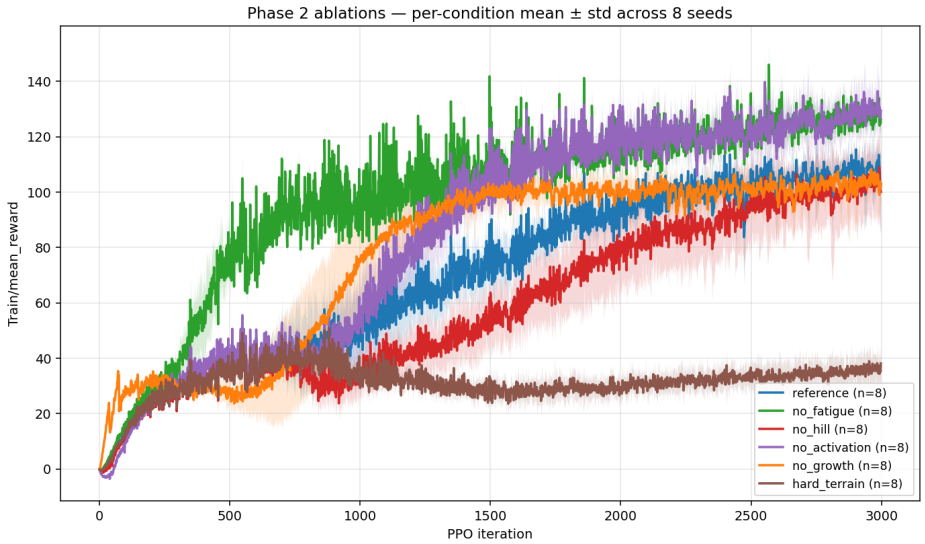

Figure 2. The training reward curves of eight seeds (from `reward_curves_overlay.png`); the shaded band is ±1 standard deviation across seeds, serving as evidence of reproduction reliability.

Chapter 4. Practical Implementation I: Reproducing and Analyzing SATA on Isaac Gym

The first stage was carried out on Isaac Gym Preview 4, in four phases. All training used 4096 parallel environments, 3000 iterations, a `[512, 256, 128]` multilayer perceptron, and the PPO algorithm, executed on the lab's NVIDIA A6000 GPU.

4.1 Phase 1: Reproduction

We trained the reference policy with SATA's released configuration. The initial three seeds yielded a mean reward of 114 ± 6; after extending to eight seeds, the value was 103.9 ± 15.9. (The same eight seeds recompute to 103.6 ± 16.0 under the matched per-term evaluation used for the cross-engine comparison in Chapter 5; the small difference is the metric, not the data.) The reproduction itself is an engineering exercise: along the way we resolved several environment and dependency issues (for example, Isaac Gym Preview 4 mandates Python 3.8; dependency versions must be pinned in a specific order, with numpy==1.21 and setuptools==59.5.0 re-pinned after the editable installs or they get overwritten; and there is the rental etiquette of a shared GPU, where the environment count must be tuned to avoid running out of memory). Only once reproduction holds does one earn the footing for subsequent analysis and critique.

4.2 Phase 2: Ablation Analysis (Eight Seeds)

We disabled each bio-inspired mechanism one at a time (single variable), trained eight seeds each, used the in-distribution scalar training reward as the metric, and ran Welch's t-test against the reference policy (using unequal variances and the Satterthwaite degrees-of-freedom approximation, because the variances across conditions differ greatly).

Condition	Mean reward ± std	Difference vs. reference	p-value
Reference	103.9 ± 15.9	—	—
No fatigue	125.8 ± 2.8	+21.9	0.006
No activation	128.3 ± 7.4	+24.4	0.003
No Hill	98.6 ± 13.4	−5.3	0.48 (not significant)
No growth	102.5 ± 7.3	−1.4	0.83 (not significant)
Hard terrain	36.0 ± 6.1	−67.9	<0.001

A counterintuitive result appears here: **disabling fatigue or activation actually raises the training reward.** But this does not mean the bio-inspired mechanisms are useless – it is precisely the symptom one should expect from "an effective sim-to-real constraint" in a clean simulator: a constraint that protects hardware will naturally sacrifice a little reward in a simulator where there is no hardware cost. Its true value can only be assessed out of distribution (Phase 3).

This phase was also a lesson in statistical rigor: the initial three-seed analysis once claimed that "only the Hill model has a clear positive contribution," but this conclusion dissolved completely under eight seeds (the apparent effect of no_hill was actually a draw from the lower tail under high variance). We therefore established the principle of not drawing conclusions from a small number of seeds.

4.3 Phase 3: The Bio-Inspired Claim and Out-of-Distribution Robustness (Core)

We evaluated all forty-eight policies (six conditions × eight seeds) under eight scenarios, for 384 evaluation cells in total. The scenarios spanned no disturbance (nominal), different payloads (5/8/10/15 kg), and different lateral external forces. The hardware baseline of the Unitree Go2 is: a body mass of about 15 kg, a rated payload of 7/8/12 kg depending on the model, and a single-joint peak torque of 45 N·m.

Core finding: the ablated policies that "won the reward" in Phase 2 won by **leaving the hardware-feasible region.**

- After disabling activation, the peak torque surged to **42.5 N·m**, approaching the Go2's 45 N·m limit (about 1.8× the simulation clipping value of 23.5).
- After disabling fatigue, walking on flat ground with no disturbance already consumed **2.5× the mechanical energy and 35× the action jerk.**

In other words, the two constraints of fatigue and activation sacrificed training reward in exchange for policies the "hardware can actually execute." An important reinterpretation: disabling activation did not actually make the actions less smooth (its jerk was in fact lower, indicating PPO smooths things on its own); its real role is to **lock down peak torque.**

Under payload, the reference policy reproduced the limitation SATA's paper acknowledges in §VI-A: as the payload increases its body height sinks (0.32 → 0.26 at 5 kg → 0.20 at 8 kg), and at the rated 8 kg its episode length drops to 2994 ± 471 steps (the large variance means some rollouts fall early). The no-fatigue policy, by contrast, beat the reference at **every** payload — reward Δ of +40 (5 kg), +94 (8 kg), +113 (10 kg), and +126 (15 kg), all $p < 0.001$ — and never fell (its episode length held near the nominal ~3578 throughout). But this advantage is the **same** sustained-high-torque behavior that shows up as 2.5× energy in nominal walking: the reference's §VI-A "failure" is the fatigue mechanism **refusing to thermally overload the actuators** to hold a beyond-rated load. In a simulator with no thermal model, refusing looks like weakness; on real hardware it is the protection the mechanism exists for. So the payload result is the core finding seen from the load axis, not a contradiction of it.

Under lateral push, the picture is different: at the training magnitude (1.5 m/s) every condition holds near nominal, and at 3–5 m/s they all collapse together. Push does **not** discriminate the bio-inspired mechanisms — the discriminating axis is sustained load, not impulse.

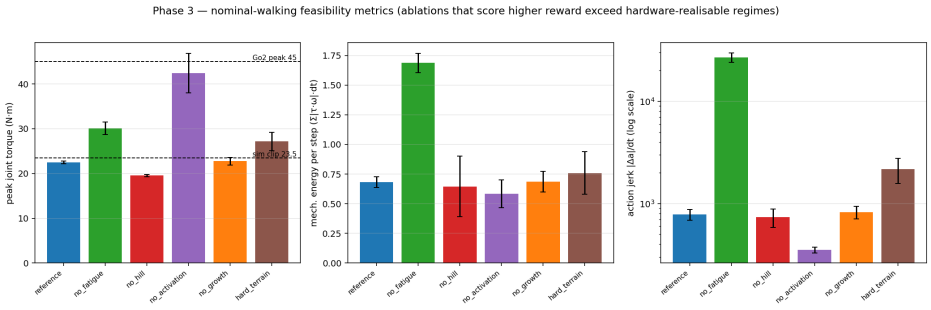

Figure 3. The hardware-feasibility chart under the no-disturbance scenario (from `nominal_feasibility.png`): peak torque / energy / action jerk for each condition, against the real-hardware reference lines.

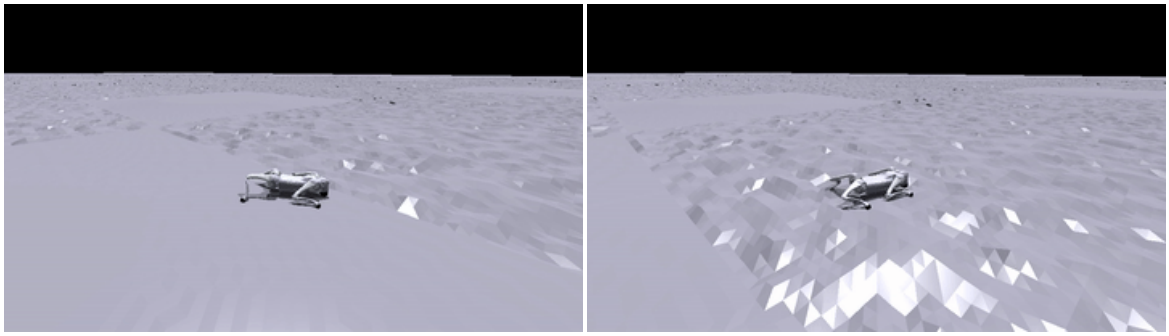

Figure 4. A comparison under a 10 kg payload (exceeding the rated value): left is the reference policy (reproducing the paper's payload limitation, gradually collapsing); right is no-fatigue (holding up, but in a hardware-infeasible way using 2.5× energy and 35× jerk). From `02_reference_payload10.gif` and `03_no_fatigue_payload10.gif`.

4.4 Phase 4: Residual Compensation

On the **frozen** reference policy, this study adds a classical "stance-phase-gated, height-based PD residual." Its compensation torque is defined as:

$$\tau_{comp} = sign \times clip(K_p \times (h^* - h) - K_d \times \dot{h},\ 0,\ \tau_{cap})$$

where h is the current body height, h* is the target height, and $\dot{h}$ is its rate of change. This residual is applied to the calf joints only when the foot is in contact (stance-phase gating), and it is added after the bio-inspired layer (an independent channel that does not pass through fatigue or Hill). The implemented parameters are: Kp = 20, Kd = 4, τ_cap = 3 N·m, h* = 0.32, sign = +1, hand-tuned on seed one under an 8 kg payload and then fixed, applied to all eight seeds. This does not retrain the policy; the goal is to test whether a simple classical add-on can recover part of the load capacity within the hardware-feasible range.

Results (two-tailed Welch's test of residual vs. reference, eight seeds): under an 8 kg payload, the reward rose from 60.5 to 82.7 (+37%), but the p-value was 0.068, **not statistically significant**; under a 10 kg payload it rose from 12.1 to 20.3 (p = 0.071, also not significant); under no disturbance it was almost identical to the reference (+0.2, p = 0.95, i.e., it does no harm when not needed). The peak torque stayed within 28 N·m (well below the 45 limit), so it indeed remained within the hardware-feasible range. An important observation is that the tuning itself was a finding: a high-gain residual (for example Kp = 80, τ_cap = 15) actually caused harm in both directions, because **the frozen policy cannot observe the residual and treats it as an unmodeled disturbance to resist;** only stance-phase gating paired with gentle gains yields a net benefit. In the end this residual recovered only about a quarter of the "no-fatigue" gap; that ceiling is precisely the consequence of "reinforcement learning and adaptive compensation not being co-designed," and it points to the future direction of co-training (such as RL2AC).

4.5 Incidental Observation: Trainability (Left as an Open Question)

On re-reading the paper's §V-A1, we noticed that the framing of its ablation is actually about **trainability** – the paper reports that "after removing the entire biomechanical model, the robot is completely unable to learn a coherent gait and only shuffles its feet asymmetrically on the ground." Out of curiosity, we ran the case of "disabling activation, Hill, and fatigue simultaneously" (keeping growth), with five seeds.

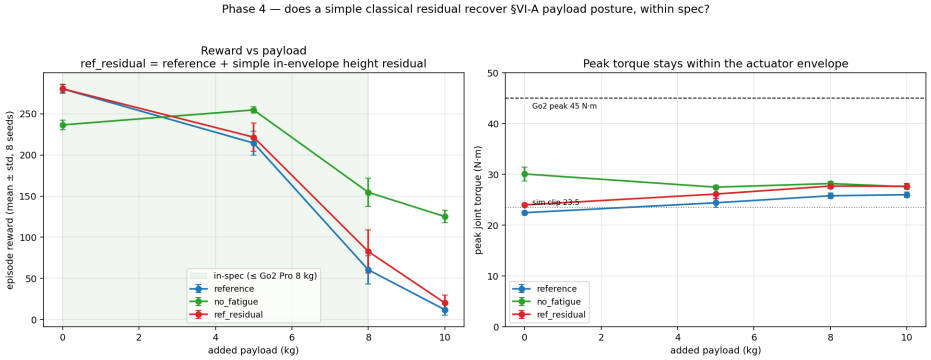

Figure 4. The reward and peak torque of the residual compensation under each payload (from `residual_payload.png`).

The result was that it still trained successfully, and the reward was even higher (138.6 ± 4.7). However, we **deliberately draw no conclusion** from this, for two reasons. First, with all three disabled, our torque pipeline degenerates into purely scaled raw torque (the $\tanh \cdot \tau$_limit activation envelope and the Hill term cancel out), which is not necessarily equivalent to the baseline the paper used (the paper did not release the code for that ablation, and its public repo ships only the full model). Second, **reward does not equal gait quality** – the paper's "cannot learn" refers to gait quality (shuffling in place), whereas we did not establish a gait-quality metric; a degenerate shuffle can still score reward. The paper also resets the robot "lying flat on the ground," whereas the public config resets to an upright low crouch, and the starting basin matters for a trainability question. This high reward therefore cannot refute the paper. We leave it as an honest open question.

Chapter 5. Practical Implementation II: Cross-Engine Migration to Isaac Lab

Isaac Gym Preview 4 has been discontinued. NVIDIA's official successor is **Isaac Lab**, built on top of Isaac Sim (and reusing the same rsl_rl PPO backend). The second stage migrates the entire SATA pipeline to Isaac Lab and answers a question that is more fundamental than the first stage: **can a faithful port still hold when only the simulation engine is swapped?**

5.1 Design Principle of the Migration

The key is to **fix the robot as the control variable**: reuse the same Go2, the same rsl_rl PPO backend, and use the Isaac Gym reproduction numbers as ground truth. If a new robot were used, one could not distinguish "a porting error" from "a genuine engine difference." This stage therefore ports as directly as possible and judges fidelity by the gap between the residual and the baseline, rather than presupposing that any inconsistency is due to a "cross-engine effect."

One key sub-migration is the control paradigm: Isaac Lab's built-in Go2 task defaults to position (PD) control, whereas SATA is a torque/force policy. Configuring it for torque control and then porting the bio-inspired layer (activation low-pass, Hill, fatigue, Gompertz growth) onto it was the bulk of the work in this stage.

5.2 Fidelity Debugging: Six Fixes

After a direct port, the policy at one point crawled along the ground with its belly down. After six fidelity fixes, the policy turned into a clean walk. One key fix was this: SATA's joint **position** limits were originally used only for the reward and termination decisions and were not written into the physics simulation; the wider built-in USD thigh limits allowed exploration to overextend, which then triggered hard-limit termination in most episodes. We deliberately preserved every intermediate failure state as a record of the debugging journey.

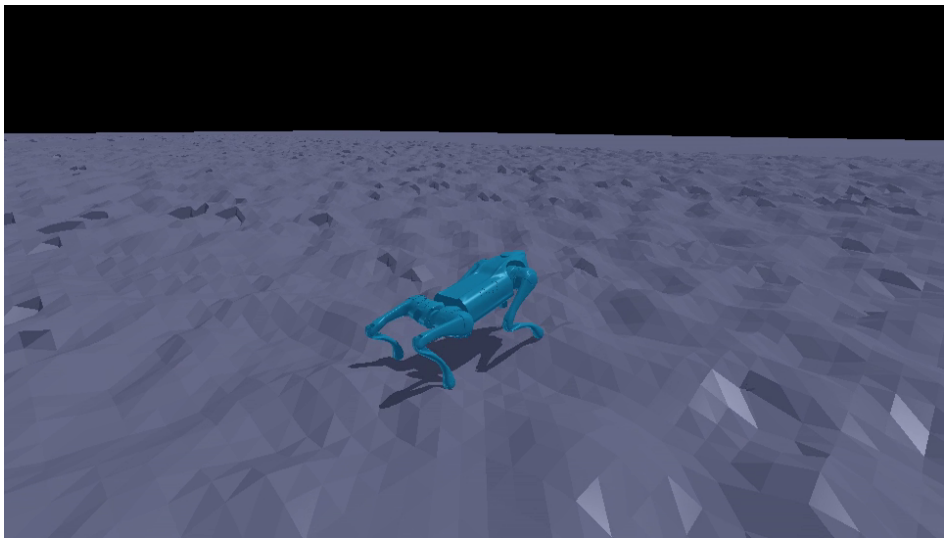

Figure 5. The cross-engine walking result: the joint trajectories trained in Isaac Lab, rendered via kinematic replay in Isaac Gym (from `06_roughReplay_cleanWalk_still.png`). (Because the RTX renderer of Isaac Sim is incompatible with the driver on this container, the gait is presented via kinematic replay.)

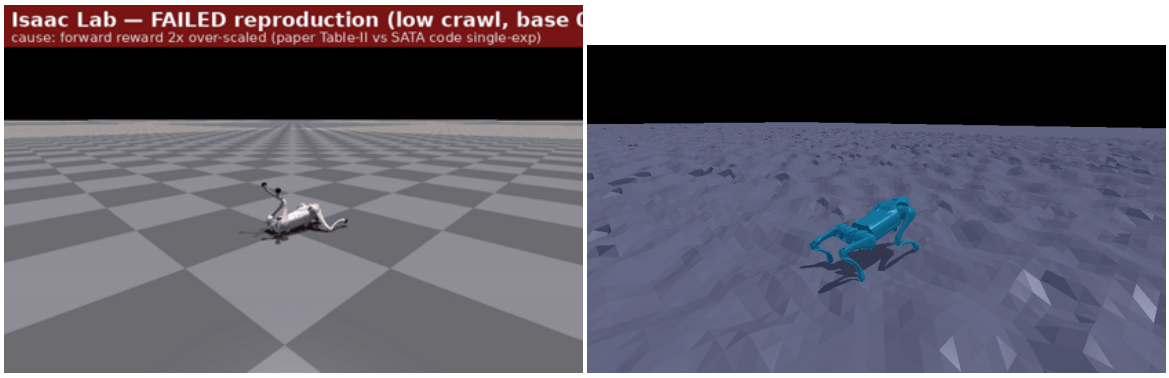

Figure 7. A comparison of the debugging journey: left is the belly-crawl caused by an early reward-configuration bug; right is the clean walk after the fix (from `01_flat_rewardBug_crawl.gif` and `06_roughReplay_cleanWalk_still.png`).

5.3 Eight-Seed Reproduction Results

We retrained eight seeds of the full bio-inspired stack on SATA's rough terrain and compared item by item against the Isaac Gym baseline:

	Mean reward (eight seeds)	Excluding one collapsed seed (seven seeds)
Isaac Lab (this study)	76.8 ± 16.2	82.3 ± 5.0
Isaac Gym (SATA baseline)	103.6 ± 16.0	108.6 ± 8.0

Both engines exhibit the **same structure**: seven seeds cluster together while one collapses late in PPO (ours s5 → 38.4, the Gym reference s7 → 68.4). We report the eight-seed numbers including the collapse as the headline result, rather than the prettier seven-seed numbers.

Removing the episode-length confound (per-step = mean return / mean episode length), the 0.76 reward ratio factors into about 10% shorter episodes × about 16% lower per-step reward. Splitting the per-step number term by term (clean seeds) is the most informative view:

Term	Gym/step	Lab/step	Lab – Gym
forward	0.0400	0.0388	−0.0012
head_height	0.0130	0.0131	+0.0001
moving_y	0.0161	0.0138	−0.0023
moving_yaw	0.0127	0.0131	+0.0004
roll	−0.0017	−0.0014	+0.0003
lin_vel_z	−0.0007	−0.0007	0.0000
fatigue	−0.0130	−0.0110	+0.0020
joint_acc	−0.0111	−0.0194	−0.0083

The positive task-reward terms (forward progress, head height, lateral movement, turning) match the Isaac Gym baseline to within about 0.002 per step – that is, the policy reproduced SATA's per-step task performance – and the per-step deficit is concentrated almost entirely (about 93%) in a single penalty term, the joint-acceleration penalty. That term's definition differs across engines: SATA uses finite differences, while we use the instantaneous acceleration from PhysX5, which captures contact-impact spikes that finite differences smooth out (we tried matching the finite-difference form; it trained markedly worse and was reverted). This **suggests** the residual is dominated by how the term is measured rather than by worse locomotion – but we have not separated "measured differently" from "genuinely jerkier," and the ~10% episode-length shortfall is left as a real, unexplained residual. We do not claim the two engines are equivalent.

Chapter 6. Results and Methodological Rigor

The credibility of this study rests on a series of deliberate methodological choices:

- **At least five seeds per condition** (Phases 2 and 3 use eight). A small number of seeds is untrustworthy in reinforcement learning – this is exactly what Henderson et al. (2018) warned against; and indeed, by extending from three to eight seeds, we overturned two preliminary conclusions.

- **Welch's t-test (unequal variances)**: the variances across conditions differ greatly, so the equal-variance assumption does not hold, and we use Welch's test with the Satterthwaite degrees-of-freedom approximation.
- **Two-tailed tests as the honest default**: Phase 4 states plainly that a one-tailed test would render the 8/10 kg results significant, but we still use the two-tailed values ($p \approx 0.07$) and report them as a "trend" rather than an "established effect."
- **Reporting the sample standard deviation (ddof=1)**, and faithfully reporting the final-iteration reward (including that late-collapsing seed).
- **"Not detected" does not equal "useless"**: the verdict on the Hill model is "no effect detected along the axes we measured," not "useless" – force-velocity shaping very likely takes effect in the high-joint-speed scenarios we did not isolate.
- **Simulation only (the biggest limitation of this project)**: there is no physical robot and no thermal model. "Within the actuator-feasible range" means "within the rated 45 N·m number," not hardware-verified. This is precisely why disabling fatigue can "win" in clean simulation – the simulator does not charge for sustained thermal overload.

We also used data to **correct two analogies from our own first draft**: fatigue is not a disturbance compensator (an external force affects all conditions equally; fatigue is a regularizer on exertion), and growth is not a runtime gain schedule (it is a training-time curriculum, already saturated at deployment – a timescale mismatch).

Chapter 7. Conclusion and Reflection

This study fully reproduced SATA's torque-based quadruped locomotion and carried out the training, evaluation, and analysis by hand on two simulation engines. Our observations can be summarized as follows: along the hardware-feasibility axis we measured, the bio-inspired mechanisms behave more like **sim-to-real feasibility constraints** than mere reward-tuning devices – disabling them does raise the reward in clean simulation, but at the cost of leaving the hardware-feasible range. A classical residual compensation term can recover part of the load capacity within the feasible range, but only to a limited degree; its ceiling stems from the lack of co-design, pointing toward co-trained reinforcement-learning and adaptive methods. As for cross-engine migration, the faithful port reproduced SATA's per-step task performance after the engine was swapped, with the remaining gap concentrated in the definitional difference of a single penalty term.

This study offers two reflections. First, statistical rigor is not a formality – we personally experienced a three-seed conclusion being overturned under eight seeds, and we therefore remain wary of "drawing conclusions from a small sample." Second, honestly knowing when not to draw a conclusion is as important as reaching one: in the incidental observation on trainability, our experimental setup had confounding factors (degeneration to raw torque, a different reset pose) and lacked a corresponding gait-quality metric, so we chose to leave it as an open question rather than overclaim. Questions of this kind – "why is torque-based reinforcement learning so hard to train" – are exactly the direction we hope to explore more deeply in the future.

References

1. Li, P., Li, H., Sun, G., Cheng, J., Yang, X., Bellegarda, G., Shafiee, M., Cao, Y., Ijspeert, A., & Sartoretti, G. (2025). *SATA: Safe and Adaptive Torque-Based Locomotion Policies Inspired by Animal Learning*. arXiv:2502.12674.
2. Hill, A. V. (1938). The heat of shortening and the dynamic constants of muscle. *Proceedings of the Royal Society of London. Series B*, 126(843), 136–195.
3. Henderson, P., Islam, R., Bachman, P., Pineau, J., Precup, D., & Meger, D. (2018). Deep Reinforcement Learning that Matters. *Proceedings of the AAAI Conference on Artificial Intelligence*, 32(1), 3207–3214.
4. Lyu, S., Lang, X., Zhao, H., Zhang, H., Ding, P., & Wang, D. (2024). *RL2AC: Reinforcement Learning-based Rapid Online Adaptive Control for Legged Robot Robust Locomotion*. Robotics: Science and Systems (RSS) 2024.
5. Chen, S., Zhang, B., Mueller, M. W., Rai, A., & Sreenath, K. (2022). *Learning Torque Control for Quadrupedal Locomotion*. arXiv:2203.05194.
6. Sood, S., et al. (2024). *DecAP: Decaying Action Priors for Accelerated Imitation Learning of Torque-Based Legged Locomotion Policies*. IROS 2024. arXiv:2310.05714.
7. Hwangbo, J., et al. (2019). Learning agile and dynamic motor skills for legged robots. *Science Robotics*, 4(26). arXiv:1901.08652.
8. Lee, J., Hwangbo, J., Wellhausen, L., Koltun, V., & Hutter, M. (2020). Learning quadrupedal locomotion over challenging terrain. *Science Robotics*, 5(47). arXiv:2010.11251.
9. Miki, T., Lee, J., Hwangbo, J., Wellhausen, L., Koltun, V., & Hutter, M. (2022). Learning robust perceptive locomotion for quadrupedal robots in the wild. *Science Robotics*, 7(62). arXiv:2201.08117.
10. Bellegarda, G., & Ijspeert, A. (2022). CPG-RL: Learning Central Pattern Generators for Quadruped Locomotion. *IEEE Robotics and Automation Letters*. arXiv:2211.00458.

Open Source

Both projects in this study are open-sourced, and anyone may use, inspect, or build upon them under their respective license terms (see the LICENSE of each project for details):

- **SATA reproduction and analysis on Isaac Gym**: https://github.com/Stanley-1013/bio-inspired-adaptive-locomotion
- **Cross-engine reproduction migrated to Isaac Lab**: https://github.com/Stanley-1013/isaaclab-torque-locomotion